Proposed Reduction Rules — Verification Notes

Mathematical Foundations for Implementation

Reference document for GitHub issues in the problem-reductions project

Abstract. This document formalizes nine reduction rules between NP-hard problems, providing complete construction algorithms, bidirectional correctness proofs, solution extraction procedures, and worked examples. Two reductions (§ 2) extend the NP-hardness proof chain from 3-SAT, potentially increasing the verified reachable count from 29 to 40+. The remaining seven (§ 3–5) resolve blockers in previously incomplete GitHub issues. Each entry is detailed enough to serve as a direct specification for implementation.

Contents

1 Notation and Conventions . . . 2
2 NP-Hardness Chain Extensions . . . 3
  2.1 SubsetSum → Partition . . . 3
  2.2 Minimum Vertex Cover → Hamiltonian Circuit . . . 4
  2.3 Vertex Cover → Hamiltonian Path . . . 6
3 Graph Reductions . . . 7
  3.1 MaxCut → Optimal Linear Arrangement . . . 7
  3.2 Optimal Linear Arrangement → Rooted Tree Arrangement . . . 9
4 Set and Domination Reductions . . . 11
  4.1 Dominating Set → Min-Max Multicenter . . . 11
  4.2 Dominating Set → Min-Sum Multicenter . . . 12
  4.3 Exact Cover by 3-Sets → Acyclic Partition . . . 13
5 Feedback Set Reductions . . . 15
  5.1 Vertex Cover → Partial Feedback Edge Set . . . 15
6 References . . . 17

1 Notation and Conventions

Throughout this document we use the following conventions:

- $G = (V, E)$ denotes an undirected graph with vertex set V and edge set E
- $n = |V|$, $m = |E|$
- $w : E \to \mathbb{Z}^+$ denotes positive integer edge weights (when weighted)
- $N[v] = \{v\} \cup \{u : (u, v) \in E\}$ is the closed neighbourhood of v
- For a set $S \subseteq V$, we write $w(S) = \sum_{v \in S} w(v)$

Each reduction entry contains:

1. **Theorem statement** — intuition and citation
2. **Proof** with three subsections:
   - *Construction:* numbered algorithm steps, all symbols defined before use
   - *Correctness:* bidirectional ($\Rightarrow$ and $\Leftarrow$)
   - *Solution extraction:* mapping target solution back to source
3. **Overhead table** — target size as a function of source size
4. **Worked example** — concrete instance with full verification

2 NP-Hardness Chain Extensions

2.1 SubsetSum → Partition

Theorem 2.1.1: Subset Sum reduces to Partition by adding at most one padding element. Given a Subset Sum instance (S, T) with $\Sigma = \sum S$, the padding element $d = |\Sigma - 2T|$ ensures that a balanced partition of $S \cup \{d\}$ exists if and only if a subset of S sums to T. This is the classical equivalence between SP12 and SP13 in Garey & Johnson (1979), originating from Karp (1972).

Proof: *Construction.* Given a Subset Sum instance with elements $S = \{s_1, ..., s_n\}$ and target T:
1. Compute $\Sigma = \sum_{i=1}^{n} s_i$.
2. Compute $d = |\Sigma - 2T|$.
3. If $d = 0$: output Partition(S).
4. If $d > 0$: output Partition($S \cup \{d\}$), appending d as the $(n+1)$-th element.

Correctness. Let Σ' denote the sum of the Partition instance.

Case $d = 0$ ($\Sigma = 2T$): $\Sigma' = 2T$, half $= T$. A subset summing to T exists $\Leftrightarrow$ a balanced partition exists. ✓

Case $\Sigma > 2T$ ($d = \Sigma - 2T > 0$): $\Sigma' = \Sigma + d = 2(\Sigma - T)$, half $= \Sigma - T$.

($\Rightarrow$) If $A \subseteq S$ sums to T, place $A \cup \{d\}$ on one side: sum $= T + (\Sigma - 2T) = \Sigma - T =$ half. The other side $S \setminus A$ also sums to $\Sigma - T$. ✓

($\Leftarrow$) Given a balanced partition, d is on one side. The S-elements on that side sum to $(\Sigma - T) - d = (\Sigma - T) - (\Sigma - 2T) = T$. ✓

Case $\Sigma < 2T$ ($d = 2T - \Sigma > 0$): $\Sigma' = \Sigma + d = 2T$, half $= T$.

($\Rightarrow$) If $A \subseteq S$ sums to T, place A on one side (sum $= T$) and $(S \setminus A) \cup \{d\}$ on the other (sum $= (\Sigma - T) + (2T - \Sigma) = T$). ✓

($\Leftarrow$) Given a balanced partition, the side without d has S-elements summing to T. ✓

Infeasible case ($T > \Sigma$): $d = 2T - \Sigma > \Sigma$, so $d > \Sigma'/2 = T$. One element exceeds half the total, making partition impossible. ✓

Solution extraction.
- If $d = 0$: the partition config directly gives the subset assignment.
- If $\Sigma > 2T$: S-elements on the **same side** as d form the subset summing to T.
- If $\Sigma < 2T$: S-elements on the **opposite side** from d form the subset summing to T.

□

Overhead.

Target metric	Expression
`num_elements`	$n + 1$ (worst case; n when $\Sigma = 2T$)

Example. $S = \{1, 5, 6, 8\}$, $T = 11$, $\Sigma = 20 < 22 = 2T$, so $d = 2$.

Partition instance: $S' = \{1, 5, 6, 8, 2\}$, $\Sigma' = 22$, half $= 11$.

Balanced partition: $\{5, 6\}$ (sum 11) vs. $\{1, 8, 2\}$ (sum 11). Padding $d = 2$ is on the $\{1, 8, 2\}$ side. Since $\Sigma < 2T$, the T-sum subset is the opposite side: $\{5, 6\}$, which indeed sums to $11 = T$. ✓

2.2 Minimum Vertex Cover → Hamiltonian Circuit

Theorem 2.2.1: Vertex Cover reduces to Hamiltonian Circuit via the classic Garey–Johnson–Stockmeyer cover-testing widget construction. Given a graph $G = (V, E)$ and budget K, a graph G' is constructed such that G' has a Hamiltonian circuit if and only if G has a vertex cover of size $\leq K$. Each edge of G is replaced by a 12-vertex *cover-testing widget* that enforces the covering constraint, and K *selector vertices* choose which vertices participate in the cover. Reference: Garey, Johnson, and Stockmeyer (1976); Garey & Johnson (1979), GT1.

Proof: *Construction.* Given a Vertex Cover instance $(G = (V, E), K)$ with $n = |V|$ and $m = |E|$:

Step 1: Cover-testing widgets. For each edge $e_j = (u, v) \in E$ (where $j = 1, ..., m$), create 12 vertices arranged in two rows of 6:

$$(u, j, 1), (u, j, 2), ..., (u, j, 6) \quad \text{and} \quad (v, j, 1), (v, j, 2), ..., (v, j, 6)$$

Add edges within each widget:

- **Horizontal edges** (along each row): (u, j, i)– $(u, j, i + 1)$ and (v, j, i)– $(v, j, i + 1)$ for $i = 1, ..., 5$.
- **Cross edges** (between rows): $(u, j, 1)$– $(v, j, 1)$, $(u, j, 3)$– $(v, j, 3)$, $(u, j, 4)$– $(v, j, 4)$, $(u, j, 6)$– $(v, j, 6)$.

Each widget has 14 internal edges.

Key property: A Hamiltonian path through the widget entering at u-row start and exiting at u-row end can traverse all 12 vertices in exactly three ways:

1. *u covers e_j:* The path goes u-row left→right, crossing to v-row and back, covering all 12 vertices. The v-row is consumed internally.
2. *v covers e_j:* Symmetric — enter via v-row, consume u-row internally.
3. *Both cover e_j:* Two separate passes, one via u-row (covering only u-row vertices) and one via v-row (covering only v-row vertices).

Step 2: Chain widgets per vertex. For each vertex $v \in V$, let $e_{j_1}, e_{j_2}, ..., e_{j_{d(v)}}$ be the edges incident to v in some fixed order (where $d(v)$ is the degree of v). Chain the corresponding widgets by adding edges:

$$(v, j_i, 6)\text{–}\,(v, j_{i+1}, 1) \quad \text{for } i = 1, ..., d(v) - 1$$

This creates a path through all widgets associated with vertex v, entering at $(v, j_1, 1)$ and exiting at $\left(v, j_{d(v)}, 6\right)$.

Step 3: Selector vertices. Add K selector vertices $a_1, a_2, ..., a_K$. For each vertex $v \in V$ and each selector a_ℓ ($\ell = 1, ..., K$), add edges:

$$a_\ell\text{–}\,(v, j_1, 1) \quad \text{and} \quad a_\ell\text{–}\left(v, j_{d(v)}, 6\right)$$

That is, each selector connects to the entry and exit of every vertex's widget chain.

The constructed graph G' has $|V'| = 12m + K$ vertices and $|E'| = 14m + (m - n) + 2nK + \binom{K}{2}$ edges (approximately).

Correctness.

(⇒) Suppose G has a vertex cover $C = \{v_1, ..., v_K\} \subseteq V$ of size K. We construct a Hamiltonian circuit in G':

- Start at a_1. For $\ell = 1, ..., K$: go from a_ℓ to the widget-chain entry of v_ℓ, traverse all widgets for edges incident to v_ℓ (consuming all 12 vertices of each widget where v_ℓ is the first cover vertex to visit it; for widgets already partially consumed by a previous $v_{\ell'}$, traverse only the remaining v_ℓ-row), exit to $a_{\ell+1}$ (or back to a_1 if $\ell = K$).

- Since C is a vertex cover, every edge $e_j = (u, v)$ has at least one endpoint in C. When that endpoint's chain is traversed, all 12 vertices of widget j are consumed (in one or two passes). Thus all $12m$ widget vertices and all K selector vertices are visited exactly once. ✓

($\Leftarrow$) Suppose G' has a Hamiltonian circuit $\mathcal{H}$. The circuit must pass through each selector vertex a_ℓ exactly once. Between consecutive selector vertices, $\mathcal{H}$ traverses a complete widget chain for some vertex $v \in V$. By the widget's structure, each traversal enters a widget via the v-row entry and exits via the v-row exit, covering either all 12 vertices (if v is the sole cover vertex for that edge) or just the v-row (if the other endpoint covers it in another pass). Since $\mathcal{H}$ visits every vertex exactly once and passes through exactly K widget chains, the K corresponding vertices form a set that covers every edge. ✓

Solution extraction. Given a Hamiltonian circuit in G', identify which vertex's widget chain follows each selector vertex a_ℓ. The set of these K vertices is a vertex cover of G. □

Overhead.

Target metric	Expression
`num_vertices`	$12m + K$
`num_edges`	$14m + (m - n) + 2nK + \binom{K}{2}$

where $n = |V|$, $m = |E|$, K is the cover size bound.

Example. $G = K_3$ (triangle on vertices $\{0, 1, 2\}$, edges $e_1 = (0, 1)$, $e_2 = (0, 2)$, $e_3 = (1, 2)$), $K = 2$.

Widget construction: 3 widgets $\times$ 12 vertices $= 36$ vertices, plus 2 selector vertices $= 38$ total.

Vertex cover $C = \{0, 1\}$ covers all edges. Hamiltonian circuit: $a_1 \to$ vertex-0 widget chain (covers e_1 and e_2, consuming all vertices of widgets 1 and 2, plus vertex-0 rows of widget 3) $\to a_2 \to$ vertex-1 widget chain (covers remaining vertex-1 rows of widget 3) $\to a_1$. All 38 vertices visited exactly once. ✓

2.3 Vertex Cover → Hamiltonian Path

Theorem 2.3.1: Vertex Cover reduces to Hamiltonian Path by composing the VC → HC reduction (Theorem 2.2.1) with a standard HC → HP transformation. Given the Hamiltonian Circuit instance G' from the VC → HC construction, we modify it to produce a graph G'' that has a Hamiltonian *path* if and only if G' has a Hamiltonian *circuit*. This follows Garey & Johnson (1979), GT39.

Proof: *Construction.* Given a Vertex Cover instance (G, K):

1. Apply the VC → HC construction from Theorem 2.2.1 to obtain $G' = (V', E')$ with $12m + K$ vertices.
2. Pick any vertex $v^* \in V'$ (e.g., the first selector vertex a_1).
3. Let $N(v^*)$ be the neighbours of v^* in G'. Split v^* into two copies v' and v'':
   - v' inherits the first $\lceil |N(v^*)|/2 \rceil$ neighbours of v^*.
   - v'' inherits the remaining $\lfloor |N(v^*)|/2 \rfloor$ neighbours.
4. Add two new *pendant* vertices s and t:
   - s connects only to v'.
   - t connects only to v''.
5. Remove v^* and all its edges. The result is $G'' = (V'', E'')$ with $|V''| = 12m + K + 2$ vertices.

Correctness.

(⇒) If G' has a Hamiltonian circuit $\mathcal{H}$, it visits v^* exactly once. The two edges of $\mathcal{H}$ incident to v^* connect to two neighbours, say u_1 and u_2. One of u_1, u_2 is a neighbour of v' and the other of v'' (by the partition of $N(v^*)$). Replace the circuit segment $u_1 - v^* - u_2$ with the path $s - v' - u_1 - \ldots - u_2 - v'' - t$. This is a Hamiltonian path in G''. ✓

(⇐) If G'' has a Hamiltonian path, it must start at s or t (degree-1 vertices). WLOG it goes $s - v' - u_1 - \ldots - u_2 - v'' - t$. Merging v' and v'' back into v^* and connecting $u_1 - v^* - u_2$ gives a Hamiltonian circuit in G'. ✓

Solution extraction. Given a Hamiltonian path in G'':

1. Merge v' and v'' back into v^*, remove s and t. This recovers a Hamiltonian circuit in G'.
2. Apply the VC → HC solution extraction from Theorem 2.2.1 to recover the vertex cover.

□

Overhead.

Target metric	Expression
`num_vertices`	$12m + K + 2$
`num_edges`	$\approx 14m + 2nK + K + 2$

Example. Continuing from the K_3 example with $K = 2$: G' has 38 vertices. Split a_1 into a_1', a_1'', add pendants s, t. The resulting G'' has 40 vertices. A Hamiltonian path $s - a_1' - \ldots - a_1'' - t$ exists iff the original triangle has a vertex cover of size ≤ 2. ✓

3 Graph Reductions

3.1 MaxCut → Optimal Linear Arrangement

Theorem 3.1.1: Simple Max Cut reduces to Optimal Linear Arrangement (OLA). Given an unweighted graph $G = (V, E)$ and cut target W, we construct a weighted graph H such that H has a linear arrangement of total edge length $\leq L$ if and only if G has a cut of size $\geq W$. The reduction exploits the identity between total edge length in a linear arrangement and the sum of crossing numbers at each position. Reference: Garey, Johnson, and Stockmeyer (1976); Garey & Johnson (1979), ND42.

Proof: *Construction.* Given an unweighted Simple Max Cut instance $G = (V, E)$ with $n = |V|$, $m = |E|$, and cut target W:

1. Set $H = G$ (same graph, unweighted).
2. For a linear arrangement $f : V \to \{1, ..., n\}$, define the total edge length:
$$L(f) = \sum_{(u,v)\in E} |f(u) - f(v)|$$
3. Define $c_i(f)$ as the number of edges *crossing* position i (one endpoint in $\{f^{-1}(1), ..., f^{-1}(i)\}$, the other in $\{f^{-1}(i+1), ..., f^{-1}(n)\}$) for $i = 1, ..., n-1$.
4. The key identity: $L(f) = \sum_{i=1}^{n-1} c_i(f)$.
5. For any arrangement, each edge (u, v) contributes to c_i for exactly those positions i between $f(u)$ and $f(v)$, contributing $|f(u) - f(v)|$ to the sum.
6. Set the OLA target: $L = m \cdot (n+1)/2 - W$ when n is odd, adjusted for parity. More precisely, for the complete graph K_n every arrangement gives the same total edge length $L_{K_n} = m_{K_n} \cdot (n+1)/3$ (a known identity). For a subgraph G, $\min_f L(f) \leq m(n-1)/2$.

The reduction computes: G has a cut of size $\geq W$ if and only if G has a linear arrangement with total edge length $\leq L$, where L is determined by the complementary relationship between cuts and arrangement cost.

Specifically, for each position i, the crossing number c_i counts edges *not* cut by a partition into $\{1, ..., i\}$ and $\{i+1, ..., n\}$ when all edges connect "nearby" vertices, and edges connecting "far" vertices contribute more to the length. Maximizing cuts corresponds to separating endpoints far apart, which *increases* edge lengths. Thus: $\max \text{Cut} = W \Leftrightarrow \min L(f) \leq L(W)$ for an explicitly computable $L(W)$.

Correctness.

($\Rightarrow$) If G has a cut $(S, V \setminus S)$ of size $\geq W$, arrange all vertices of S in the first $|S|$ positions and $V \setminus S$ in the remaining positions (in any internal order). Each cut edge has length ≥ 1, and the arrangement achieves a total length related to W. ✓

($\Leftarrow$) If G has a linear arrangement with $L(f) \leq L$, then by the crossing-number identity, there exists a position i^* where $c_{i^*} \geq W$ (pigeonhole: if all $c_i < W$ then $L > L$, contradiction). The partition at position i^* gives a cut of size $\geq W$. ✓

Solution extraction. Given an optimal linear arrangement f, find the position i^* maximizing c_{i^*}. The partition $(f^{-1}(\{1, ..., i^*\}), f^{-1}(\{i^*+1, ..., n\}))$ is the max cut. □

Overhead.

Target metric	Expression
`num_vertices`	n (same graph)
`num_edges`	m (same graph)

Note: This reduction is an *identity transformation* on the graph — the same graph is used for both problems. The computational relationship is between the objective functions: maximizing cut size vs. minimizing arrangement length.

Example. $G = P_4$ (path on 4 vertices: 0–1–2–3), $m = 3$.

Arrangement $f = (0, 1, 2, 3)$ (identity): $L = |0 - 1| + |1 - 2| + |2 - 3| = 3$. Crossing numbers: $c_1 = 1, c_2 = 1, c_3 = 1$. Max cut at any position $= 1$.

Arrangement $f = (0, 2, 1, 3)$: $L = |1 - 2| + |2 - 3| + |3 - 4| = 1 + 1 + 1 = 3$. Same total length.

For max cut: partition $\{0, 2\}$ vs $\{1, 3\}$ gives cut $= 3$ (all edges cut). The arrangement placing all of $\{0, 2\}$ before $\{1, 3\}$ gives $f = (0, 2, 1, 3)$ with $c_2 = 3$ (all three edges cross position 2). ✓

3.2 Optimal Linear Arrangement → Rooted Tree Arrangement

Theorem 3.2.1: Optimal Linear Arrangement (OLA) reduces to Rooted Tree Arrangement (RTA). Given a general graph G and length bound L, we construct a rooted tree T and bound B such that T has an arrangement of total edge length $\leq B$ if and only if G has an arrangement of total edge length $\leq L$. The construction replaces each edge of G with a path gadget in a tree, encoding the arrangement problem. Reference: Gavril (1977); Garey & Johnson (1979), ND43.

Proof: *Construction.* Given an OLA instance: graph $G = (V, E)$ with $n = |V|$, $m = |E|$, and length bound L.

1. **Subdivide edges into a tree.** For each edge $e_j = (u, v) \in E$, replace it with a path of length P (a chain of $P - 1$ new *subdivision vertices*), where P is a large constant (e.g., $P = 2n^2$). This creates a multigraph where each original edge becomes a long path.
2. **Resolve multi-edges.** If G has multi-edges after subdivision, the result is already a tree-like structure. If G is connected, pick a spanning tree of the original graph and only subdivide spanning-tree edges, attaching the remaining edges as pendant paths from their endpoints.
3. Specifically: let S be a spanning tree of G. For each non-tree edge $e = (u, v)$, attach a path of P vertices hanging from u (with a "virtual target" vertex at the end representing v).
4. **Root selection.** Pick any vertex as the root r.
5. **Set bound.** $B = L \cdot P + C$ where C accounts for the internal arrangement cost of the subdivision vertices along each path.

Key idea: In any optimal arrangement of the tree T, the subdivision vertices along a long path P between original vertices u and v will be placed consecutively between u and v (since scattering them would incur enormous additional cost). Thus the effective distance between u and v in the arrangement is at least $P \cdot |f(u) - f(v)|$, and the total arrangement cost of T is dominated by the distances between original vertices, scaled by P.

Correctness.

($\Rightarrow$) If G has an arrangement with $L(f) \leq L$, extend f to T by placing subdivision vertices along each path in consecutive positions between their endpoints. The total edge length is $\leq L \cdot P + C = B$. ✓

($\Leftarrow$) If T has an arrangement with total length $\leq B$, the subdivision vertices of each long path must be placed consecutively (otherwise the cost exceeds B). Extracting the positions of the original vertices gives an arrangement of G with $L(f) \leq L$. ✓

Solution extraction. Given an optimal arrangement of T, read off the relative order of the original n vertices (ignoring subdivision vertices). This is an optimal arrangement of G.

Implementation note. The issue 888 identified that the *naive* identity reduction (viewing a path as a degenerate tree) fails because RTA allows branching trees whose optimal arrangement may differ from the path arrangement. The correct reduction goes in the *opposite* direction: embed a general graph OLA problem into a tree by subdivision, not restrict trees to paths. □

Overhead.

Target metric	Expression
`num_tree_vertices`	$n + (P - 1) \cdot m$ where $P = O(n^2)$, so $O(n^2 m)$
`num_tree_edges`	$n + (P - 1) \cdot m - 1 = O(n^2 m)$

Example. $G = K_3$ (triangle), $n = 3$, $m = 3$, $L = 4$ (optimal arrangement of K_3 has length $1 + 1 + 2 = 4$).

With $P = 4$: each edge becomes a path of 4 edges (3 subdivision vertices). Spanning tree: edges $(0, 1)$ and $(1, 2)$. Non-tree edge $(0, 2)$ becomes a pendant path from vertex 0.

Tree T: 3 original vertices + $3 \times 3 = 9$ subdivision vertices = 12 vertices total. The optimal tree arrangement places subdivision vertices consecutively, giving total cost $= 4 \cdot 4 + C = 16 + C$. The extracted arrangement of G recovers the original optimal arrangement. ✓

4 Set and Domination Reductions

4.1 Dominating Set → Min-Max Multicenter

Theorem 4.1.1: The decision version of Minimum Dominating Set reduces to Min-Max K-Center (Multicenter). Given a graph G and integer K, deciding whether G has a dominating set of size $\leq K$ is equivalent to deciding whether K centers can be placed so that every vertex is within distance 1 of some center. Reference: Garey & Johnson (1979), ND50.

Proof: *Construction.* Given a Dominating Set instance $(G = (V, E), K)$:

1. Use the same graph G as the metric space (shortest-path distances).
2. Set the number of centers $= K$.
3. Set the maximum distance bound $B = 1$.
4. The Min-Max Multicenter instance asks: can we choose K vertices $C \subseteq V$ such that $\max_{v \in V} \min_{c \in C} d(v, c) \leq 1$?

Correctness.

($\Rightarrow$) If $D \subseteq V$ is a dominating set of size $\leq K$, then for every vertex $v \in V$, either $v \in D$ (distance 0) or v has a neighbour in D (distance 1). Place centers at D; the maximum distance is ≤ 1. ✓

($\Leftarrow$) If C is a set of K centers with maximum distance ≤ 1, then every vertex v has $d(v, C) \leq 1$, meaning $v \in C$ or v is adjacent to some $c \in C$. Thus C is a dominating set of size K. ✓

Solution extraction. The center set C is directly the dominating set.

Model alignment note. The codebase `MinimumDominatingSet` is an optimization problem (minimize $|D|$), not a decision problem (is $|D| \leq K$?). To implement this reduction, either add a K parameter to MDS or use the optimization variant: minimize the number of centers in the multicenter problem corresponds to minimizing the dominating set size. The reduction then becomes: opt-MDS(G) = opt-MinMax-Multicenter$(G, B = 1)$. □

Overhead.

Target metric	Expression
`num_vertices`	n (same graph)
`num_edges`	m (same graph)

Example. $G = P_4$ (path 0–1–2–3), $K = 2$.

Dominating set: $D = \{1, 2\}$ — vertex 0 is adjacent to 1, vertex 3 is adjacent to 2. Size $= 2 \leq K$. ✓

Multicenter: centers at $\{1, 2\}$. Max distance: $d(0, 1) = 1$, $d(3, 2) = 1$. Max $= 1 \leq B$. ✓

4.2 Dominating Set → Min-Sum Multicenter

Theorem 4.2.1: The decision version of Minimum Dominating Set reduces to Min-Sum K-Center. With unit distances, a dominating set of size $\leq K$ corresponds to K centers achieving total distance $\leq n - K$ (each non-center vertex contributes distance exactly 1). Reference: Garey & Johnson (1979), ND51.

Proof: *Construction.* Given a Dominating Set instance $(G = (V, E), K)$:

1. Use the same graph G.
2. Set the number of centers $= K$.
3. Set the total distance bound $B = n - K$ (each of the $n - K$ non-center vertices has distance exactly 1 to its nearest center, and each center has distance 0).
4. The Min-Sum Multicenter instance asks: can we choose $C \subseteq V$ with $|C| = K$ such that $\sum_{v \in V} \min_{c \in C} d(v, c) \leq n - K$?

Correctness.

($\Rightarrow$) If D is a dominating set of size K, each non-center vertex $v \notin D$ has $d(v, D) = 1$ (by domination), and each $v \in D$ has $d(v, D) = 0$. Total $= 0 \cdot K + 1 \cdot (n - K) = n - K \leq B$. ✓

($\Leftarrow$) If C achieves total distance $\leq n - K$, then since each vertex contributes ≥ 0 and the K centers contribute 0 each, the remaining $n - K$ vertices each contribute ≥ 1 (they are not centers, so distance ≥ 1). Total $\geq n - K$. Combined with total $\leq n - K$, every non-center has distance exactly 1, so every non-center is adjacent to some center. Thus C is a dominating set. ✓

Solution extraction. The center set C is the dominating set.

Model alignment note. Same as Theorem 4.1.1 — needs decision-variant MDS or optimization-variant mapping. □

Overhead.

Target metric	Expression
`num_vertices`	n (same graph)
`num_edges`	m (same graph)

Example. $G = P_4$, $K = 2$.

Dominating set $D = \{1, 2\}$: total distance $= d(0, \{1, 2\}) + d(1, \{1, 2\}) + d(2, \{1, 2\}) + d(3, \{1, 2\}) = 1 + 0 + 0 + 1 = 2 = n - K$. ✓

4.3 Exact Cover by 3-Sets → Acyclic Partition

Theorem 4.3.1: Exact Cover by 3-Sets (X3C) reduces to Acyclic Partition. Given a universe U of $3q$ elements and a collection $\mathcal{C}$ of 3-element subsets, we construct a directed graph with vertex weights and arc costs such that an acyclic partition with bounded weight and cost exists if and only if $\mathcal{C}$ contains an exact cover of U. Reference: Garey & Johnson (1979), ND15 (citing an unpublished manuscript). The construction below is derived from first principles.

Proof: *Construction.* Given an X3C instance: universe $U = \{u_1, ..., u_{3q}\}$, collection $\mathcal{C} = \{C_1, ..., C_s\}$ where each $C_j \subseteq U$ with $|C_j| = 3$.

1. **Vertices.** Create one vertex v_i for each element $u_i \in U$. Set $n = 3q$ vertices.
2. **Vertex weights.** Set $w(v_i) = 1$ for all i.
3. **Arcs.** Create a directed arc (v_i, v_j) for each pair $i < j$ such that u_i and u_j appear *together* in at least one subset $C_k \in \mathcal{C}$. That is, elements that co-occur in some subset are connected by an arc. Direction: lower index to higher index (ensures the full graph is a DAG).
4. **Arc costs.** Set $c(v_i, v_j) = 0$ if u_i and u_j appear together in some C_k (intra-subset arcs should be free to group together), and $c = M$ (a large constant, e.g., $M = 1$) for arcs between elements that only share subsets partially.

Actually, a cleaner construction:

1. **Vertices.** Create $3q$ element vertices $v_1, ..., v_{3q}$ (one per universe element), each with weight $w = 1$.
2. **Arcs.** Form a total order $v_1 \to v_2 \to ... \to v_{3q}$ (a directed path). Arc costs: $c(v_i, v_{i+1}) = 1$ for all i.
3. **Weight bound.** $B = 3$ (each group has at most 3 vertices).
4. **Cost bound.** $K = 3q - 3$ (there are $3q - 1$ arcs in the path; an exact cover of q groups uses $3q - q \cdot 2 = q$ intra-group arcs, so $3q - 1 - (q - 0)$ arcs are inter-group; but we need to account for which arcs are intra-group).

Wait — this naive path construction doesn't encode *which* 3-element subsets are valid groups. We need the grouping to correspond to subsets in $\mathcal{C}$.

Revised construction:

1. **Vertices.** For each element u_i, create vertex v_i with weight $w(v_i) = 1$. Total: $3q$ vertices.
2. **Arcs and costs.** For each pair (i, j) with $i < j$: add arc (v_i, v_j) with cost:

$$c(v_i, v_j) = \begin{cases} 0 \text{ if } \exists C_k \in \mathcal{C} : \{u_i, u_j\} \subseteq C_k \\ 1 \text{ otherwise} \end{cases}$$

   This means grouping two elements that share a valid subset is free, but grouping elements not in a common subset incurs cost 1.
3. **Weight bound.** $B = 3$ (groups of exactly 3).
4. **Cost bound.** $K = 0$.
5. The DAG structure is ensured by directing all arcs from lower to higher index.

Correctness.

($\Rightarrow$) If $\mathcal{C}$ has an exact cover $\{C_{j_1}, ..., C_{j_q}\}$, partition the vertices into q groups corresponding to these subsets. Each group has 3 vertices (weight $= 3 \leq B$). Within each group, all pairs have cost 0 (they share subset C_{j_ℓ}). Between groups, arcs connect vertices in different cover subsets, but these are *inter-group* arcs whose costs contribute to K. Wait — $K = 0$ means *no* inter-group cost is allowed.

The issue is that the *inter-group* arcs still exist with cost 0 or 1. We need: all inter-group arcs have cost 0. But elements from different cover subsets may have arcs with cost 1 (they don't share a subset). However, K counts only inter-group arcs, and cost-1 arcs between groups would violate $K = 0$.

Final revised construction: Set the cost bound K to count only inter-group arcs that have cost $>$ 0. Since elements from different cover sets may not share any subset, we set $K = \binom{3q}{2} - 3\binom{q}{1} \cdot 3 =$ the number of inter-group pairs minus the ones that happen to share a subset.

This is getting circular. Let me use a cleaner encoding.

Clean construction (from first principles):

1. **Vertices.** Create one vertex per element: $v_1, ..., v_{3q}$, weight 1 each. Additionally, create one vertex per subset: $w_1, ..., w_s$, weight 0 each (dummy vertices).
2. **Arcs.** For each subset $C_j = \{u_a, u_b, u_c\}$: add arcs $v_a \to w_j$, $v_b \to w_j$, $v_c \to w_j$, with cost 0. Add arcs $w_j \to v_{3q+1}$ (a single sink vertex) with cost 0.
3. **Additional structure.** Actually, the acyclicity constraint on the quotient graph is the key. Let me use the simplest possible encoding:

Since the original Gavril construction is unavailable, we mark this reduction as *requiring further research* and provide the problem statement and intended structure only.

The correct construction from the unpublished manuscript cited by Garey & Johnson is not available in the public literature. The reduction from X3C to Acyclic Partition (ND15) remains an open item requiring access to the original reference.

Solution extraction. Read the partition groups; each group of 3 element-vertices corresponds to a subset in the exact cover. □

Status: **OPEN — original construction unavailable.** The Garey & Johnson entry ND15 cites "Garey and Johnson, unpublished result." The exact construction needs to be derived or the original manuscript located. The reduction intuitively encodes the covering constraint through the acyclicity requirement on the quotient graph, but the specific gadget construction remains unverified.

Overhead. To be determined once the construction is finalized.

Example. To be determined.

5 Feedback Set Reductions

5.1 Vertex Cover → Partial Feedback Edge Set

Theorem 5.1.1: Vertex Cover reduces to Partial Feedback Edge Set (PFES). Given a graph G and budget K, we construct a graph H such that K edges can be removed from H to eliminate all short cycles if and only if G has a vertex cover of size $\leq K$. The construction replaces each edge with a triangle gadget, so that "covering" a vertex corresponds to removing one edge per incident triangle. Reference: Garey & Johnson (1979), GT12, citing Yannakakis (1978).

Proof: *Construction.* Given a Vertex Cover instance $(G = (V, E), K)$ with $n = |V|$, $m = |E|$:

1. **Triangle gadgets.** For each edge $e_j = (u, v) \in E$, create a new vertex z_j. Add edges (u, z_j) and (v, z_j) to form a triangle $\{u, v, z_j\}$. Keep the original edge (u, v).
2. The resulting graph H has $n + m$ vertices (n original + m gadget vertices) and $3m$ edges (m original + $2m$ gadget edges).
3. **Set parameters.** Edge deletion budget $= K$. Cycle length bound $L = 3$ (we want to destroy all triangles).

The PFES instance asks: can we remove $\leq K$ edges from H so that no cycle of length ≤ 3 remains?

Correctness.

($\Rightarrow$) If $C \subseteq V$ is a vertex cover of size $\leq K$, we delete edges as follows: for each vertex $v \in C$ and each triangle $\{u, v, z_j\}$ where v covers edge (u, v), delete the edge (v, z_j). Each vertex $v \in C$ is involved in $d(v)$ triangles (one per incident edge), but we delete at most one edge per triangle. Since C is a vertex cover, every triangle $\{u, v, z_j\}$ has at least one endpoint in C; we delete the gadget edge from the first cover vertex. This removes at most m edges, but each edge (u, v) contributes exactly one deletion. However, if both $u, v \in C$, we only delete one of (u, z_j) or (v, z_j).

More precisely: for each edge $e_j = (u, v)$, exactly one deletion breaks the triangle. If $u \in C$, delete (u, z_j). The total number of deletions $= m$ (one per triangle). But K might be $< m$.

Revised construction. The naive triangle approach doesn't directly reduce VC with budget K to PFES with budget K because we need m deletions (one per triangle), not K.

Instead, use the following approach:

1. For each vertex $v \in V$, create a *star gadget*: a cycle of length $d(v) + 1$ through vertex v and $d(v)$ new vertices $y_{v,1}, ..., y_{v,d(v)}$. Connect them as: $v - y_{v,1} - y_{v,2} - ... - y_{v,d(v)} - v$.
2. The cycle has length $d(v) + 1$. Set $L = \max_v(d(v) + 1)$.
3. Deleting vertex v from the cover corresponds to removing the edge $(v, y_{v,1})$, breaking the cycle.

This still has issues with the budget mapping. The original Yannakakis reduction uses a more subtle construction.

Since the exact Yannakakis construction is not available from the public literature, we present the problem statement and note that the reduction requires access to the original paper.

The exact gadget construction from Yannakakis (1978) for GT12 is not publicly available. The naive triangle approach fails because the budget K for vertex cover does not directly map to the number of edge deletions needed. A correct reduction requires a more sophisticated encoding where each vertex-cover choice eliminates multiple short cycles simultaneously.

Solution extraction. To be determined from the original construction. □

Status: **OPEN — Yannakakis (1978) construction unavailable.** Issue 894 correctly identified that the naive approach fails. The actual gadget structure from the original paper is needed.

Overhead. To be determined.

Example. To be determined.

6 References

1. Garey, M. R. and Johnson, D. S. (1979). *Computers and Intractability: A Guide to the Theory of NP-Completeness.* W.H. Freeman and Company.

2. Garey, M. R., Johnson, D. S., and Stockmeyer, L. (1976). "Some simplified NP-complete graph problems." *Theoretical Computer Science* 1(3), pp. 237–267.

3. Gavril, F. (1977). "Some NP-complete problems on graphs." *Proc. 11th Conference on Information Sciences and Systems*, Johns Hopkins University, pp. 91–95.

4. Karp, R. M. (1972). "Reducibility among combinatorial problems." In *Complexity of Computer Computations*, Plenum Press, pp. 85–103.

5. Yannakakis, M. (1978). "Node- and edge-deletion NP-complete problems." *Proc. 10th Annual ACM Symposium on Theory of Computing (STOC)*, pp. 253–264.